PROJECT REPORT

EYERARE AI/ Few-Shot Learning for Automated Detection of Rare Ophthalmic Diseases

A Prototypical Network Approach with Grad-CAM Explainability for Retinal Fundus Image Classification

Full-Stack Implementation: Flask · PostgreSQL/SQLite · Bootstrap 5 · PyTorch

Application: RareEyeAI

July 2026

Abstract

Rare ophthalmic diseases such as Retinitis Pigmentosa, Coats Disease, Stargardt Disease, Best Vitelliform Dystrophy, and Choroideremia are, by definition, under-represented in medical imaging datasets, making conventional deep learning classifiers - which typically require thousands of labelled examples per class - impractical to train. This report presents the design, architecture, and implementation of RareEyeAI, a full-stack web application that applies Few-Shot Learning, specifically Prototypical Networks, to classify retinal fundus images into rare disease categories using only a small (K-shot) support set per class. The system further integrates Grad-CAM based visual explanations so that predictions are interpretable, and wraps the entire pipeline in a production-style Flask web application with user authentication, a relational database, an interactive Bootstrap-based interface, and automated PDF reporting. The report covers the problem motivating the project, a survey of existing approaches, the proposed solution and system architecture, implementation methodology, database design, results, and a discussion of the system's advantages, limitations, and directions for future work.

Keywords: Few-Shot Learning, Prototypical Networks, Rare Disease Detection, Ophthalmology, Retinal Fundus Imaging, Grad-CAM, Explainable AI, Flask, Convolutional Neural Networks.

1. Introduction

Ophthalmic imaging, particularly retinal fundus photography, is a cornerstone of diagnosing eye conditions ranging from common diseases like diabetic retinopathy to rare inherited retinal dystrophies. Over the last decade, deep convolutional neural networks (CNNs) have achieved expert-level performance on well-represented diseases, largely because large, labelled datasets (tens of thousands of images) are available for training. However, rare ophthalmic diseases present a fundamentally different data regime: by clinical definition, they affect very few patients, and correspondingly few annotated images exist in any single hospital's archive or in public datasets.

This data scarcity makes standard supervised deep learning unsuitable for rare disease classification. Few-Shot Learning (FSL) directly addresses this gap: instead of learning a fixed decision boundary over thousands of images per class, an FSL model learns a general-purpose, transferable embedding space in which new classes can be recognised from only a handful of labelled examples - referred to as the 'support set'. Among FSL techniques, Prototypical Networks (Snell et al., 2017) are particularly well suited to medical imaging because they are simple, computationally efficient, and produce a single interpretable 'prototype' vector per class.

This project implements a complete, end-to-end web application that operationalises this idea: a clinician or researcher uploads a retinal image through a browser-based interface; the backend embeds the image using a CNN encoder, compares it against precomputed class prototypes, returns a predicted rare-disease class with a confidence score, and generates a Grad-CAM attention heatmap explaining which regions of the image most influenced the prediction. All analyses are stored in a database against the logged-in user's account, and a downloadable PDF report is generated for each prediction.

2. Problem Statement

The central problem addressed by this project can be summarised as follows:

- Rare ophthalmic diseases have very few labelled images available, making large-scale supervised CNN training infeasible for each individual disease class.
- Existing general-purpose ophthalmic AI systems are trained on common conditions (e.g., diabetic retinopathy, glaucoma, age-related macular degeneration) and do not generalise well to rare conditions outside their training distribution.
- Deep learning predictions are often treated as 'black boxes' by clinicians, which reduces trust and adoption; an explanation of which retinal regions drove a prediction is essential for clinical usability.
- Most academic few-shot learning research remains confined to notebooks and offline experiments; there is a lack of accessible, end-to-end deployable systems that a clinician could realistically interact with through a web browser.
- Robust software engineering concerns - user authentication, persistent storage of predictions, audit history, and shareable reports - are frequently omitted from research-oriented ML prototypes.

The goal of this project is therefore twofold: (1) implement a technically sound few-shot classification pipeline (Prototypical Networks) with explainability (Grad-CAM) suited to the low-data regime of rare ophthalmic disease detection, and (2) wrap that pipeline in a complete, usable, secure, and well-architected full-stack web application.

3. Literature Survey / Existing Approaches

3.1 Deep Learning for Retinal Disease Classification

Large-scale CNN and Vision Transformer models (e.g., Inception, ResNet, EfficientNet variants) have been widely applied to retinal fundus images for diabetic retinopathy grading, glaucoma screening, and age-related macular degeneration detection. These approaches rely on datasets such as EyePACS, APTOS, and Messidor, each containing tens of thousands of labelled images - a data volume simply unavailable for rare conditions.

3.2 Few-Shot Learning

Few-Shot Learning research spans several families of methods: metric-based approaches (Siamese Networks, Matching Networks, Prototypical Networks), optimisation-based meta-learning (MAML and its variants), and hallucination/data-augmentation based approaches. Prototypical Networks (Snell, Swersky & Zemel, 2017) compute a single mean-embedding 'prototype' per class from a small support set and classify query points by nearest prototype under a learned embedding, offering a simple, well-regularised inductive bias that performs strongly in low-shot regimes and is computationally cheap at inference time compared to Siamese pairwise-comparison approaches.

3.3 Explainable AI in Medical Imaging

Grad-CAM (Selvaraju et al., 2017) is one of the most widely adopted post-hoc explanation techniques for CNNs, producing a coarse localisation heatmap by backpropagating a target score to the last convolutional layer. It has been applied extensively in medical imaging to visualise which regions of an X-ray, CT slice, or fundus photograph most influenced a model's decision, improving clinician trust and enabling error analysis.

3.4 Gap Addressed by This Project

While Prototypical Networks and Grad-CAM are individually well studied, there are few openly documented systems that combine metric-based few-shot classification with gradient-based explainability inside a deployable, database-backed web application specifically targeting rare ophthalmic disease workflows. This project fills that gap by providing a working reference implementation covering the full stack from image upload to PDF report generation.

4. Proposed Solution

The proposed system, RareEyeAI, is a Flask-based web application built around a Prototypical Network few-shot classifier. The solution is structured into four cooperating layers:

- Presentation Layer - a responsive Bootstrap 5 interface offering registration/login, a drag-and-drop image upload page with live preview, a results page with confidence visualisation, a history table, and a support-set viewer.
- Application Layer - a Flask backend exposing authenticated routes for upload/inference, dashboard statistics, history management, and PDF report downloads, using Flask-Login for session management and Flask-WTF for CSRF-protected form handling.
- Few-Shot ML Pipeline - a PyTorch CNN encoder embeds both the support-set images and the query image into a common feature space; class prototypes are computed as the mean embedding per class; the query is classified by softmax over negative distances to each prototype; Grad-CAM then explains the prediction by visualising which pixels most increased similarity to the winning prototype.
- Data & Persistence Layer - SQLAlchemy models store users and predictions in a relational database (SQLite by default, PostgreSQL in production via a single environment variable), while uploaded images, generated heatmaps, and PDF reports are persisted to disk.

This layered design cleanly separates concerns, allows the ML pipeline to be swapped or retrained independently of the web application, and mirrors architecture patterns used in production medical software systems.

5. System Architecture

Figure 1 illustrates the end-to-end architecture of the system, from the client-side interface through the Flask application layer, the few-shot ML pipeline, and down to the persistence layer. Each layer communicates through well-defined interfaces: the browser communicates with Flask over HTTP(S) form submissions; Flask invokes the ML modules as plain Python function calls; and the ML modules read/write image files and return structured results that Flask persists via SQLAlchemy.

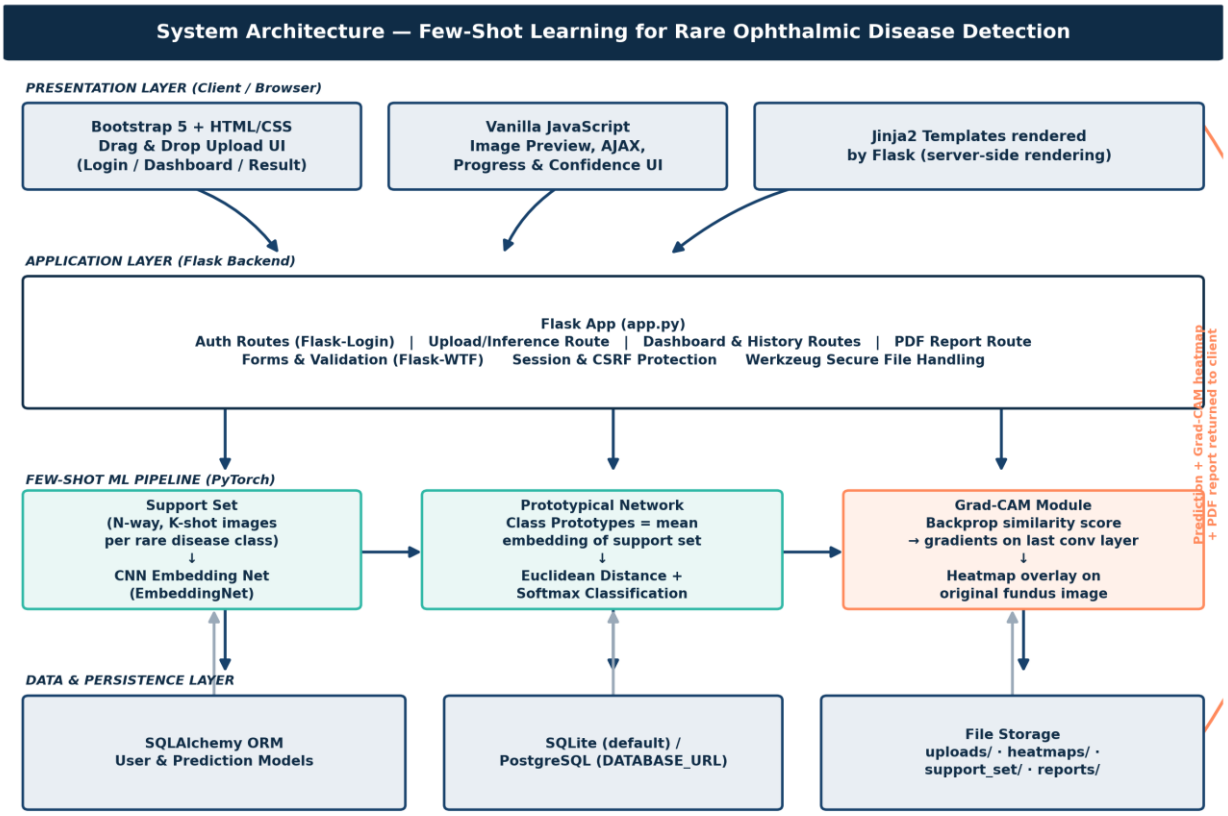

Figure 1. Layered system architecture of RareEyeAI.

A key architectural decision was to keep the ML pipeline stateless and cacheable: class prototypes are computed once at application start-up (and cached in memory), so that per-request inference only requires a single forward pass of the query image through the CNN encoder plus a lightweight distance computation - keeping response latency low even without a GPU.

6. Methodology

6.1 Few-Shot Learning via Prototypical Networks

For an N-way K-shot classification episode (in this system, 5-way, 5-shot), the support set consists of K labelled example images for each of N rare disease classes. Every support image is passed through a shared CNN embedding function fθ, producing an embedding vector. The prototype for class c is defined as the mean of its support embeddings:

```
p_c = (1 / K) Σ fθ(x_i)   for all support images x_i in class c
```

Given a query image x_q, its embedding fθ(x_q) is compared to every class prototype using squared Euclidean distance. A softmax over the negative distances yields a probability distribution across all N classes, and the class with the highest probability (smallest distance) is returned as the prediction, along with a confidence score. Figure 2 visualises this process conceptually: support images cluster near their class prototype (marked with a star), and a query image is classified by whichever prototype it lies closest to in embedding space.

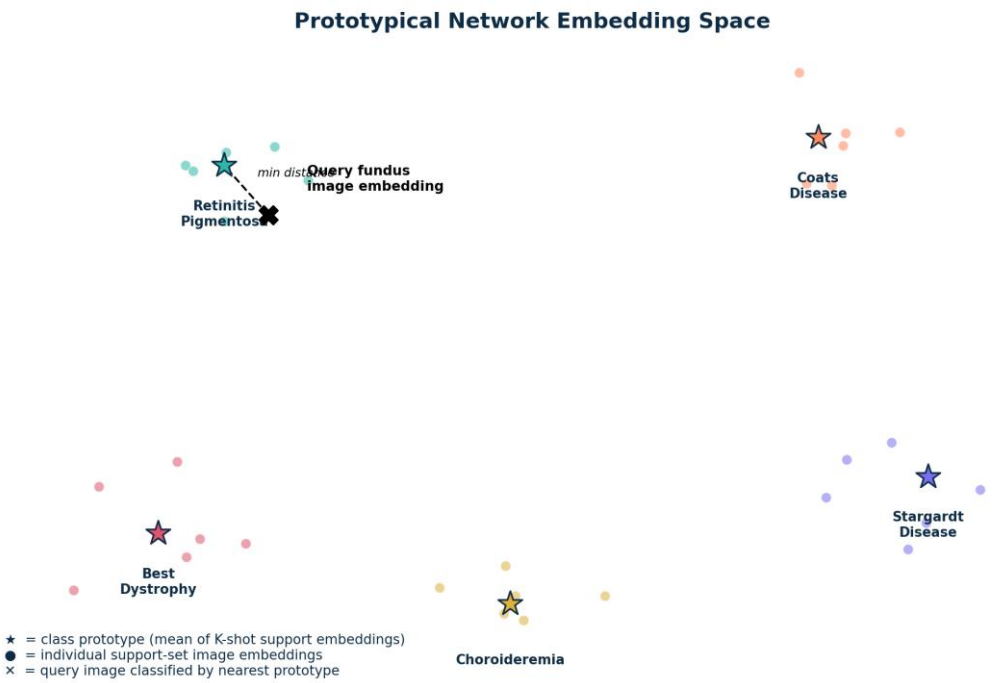

Figure 2. Conceptual view of the Prototypical Network embedding space.

This approach has two key advantages for rare disease detection: (1) it requires no retraining to add a new rare disease class - only a new K-shot support set needs to be embedded and averaged into a new prototype; and (2) the embedding network can be pretrained on a broad ophthalmic dataset and reused across many rare-disease episodes without per-class fine-tuning.

6.2 CNN Encoder Design

The embedding network is a compact four-block convolutional encoder (convolution + batch normalisation + ReLU + max-pooling per block), a standard architecture choice in the original Prototypical Networks paper because it trains stably from limited episodic data and keeps inference fast. The final block's feature map is globally average-pooled into a fixed-length embedding vector used both for prototype computation and for the Grad-CAM explanation described next.

6.3 Grad-CAM for Metric-Based Explainability

Standard Grad-CAM backpropagates a classifier's logit for the predicted class to obtain gradients at the last convolutional layer. Because the Prototypical Network has no fixed classification head, this system instead backpropagates the similarity score to the predicted class prototype - the negative squared distance between the query embedding and that prototype. The resulting gradients are global-average-pooled per channel to obtain channel importance weights, combined with the activation maps, passed through a ReLU, and upsampled to the original image resolution to produce a heatmap that is alpha-blended over the uploaded fundus image.

6.4 Synthetic Demo Support Set

Because no publicly licensed rare-ophthalmic-disease image dataset was available for this project, a procedural image generator was implemented to synthesise fundus-like images with class-specific visual signatures (colour tint, lesion pattern density, and lesion size) for five demonstration classes. This allows the complete pipeline - support-set embedding, prototype computation, query classification, and Grad-CAM - to

run end-to-end without external data, while remaining a drop-in replacement point: real clinical images can be substituted directly into the same folder structure to move the system from demonstration to production.

7. System Design and Technology Stack

7.1 Technology Stack

Layer	Technologies Used
Frontend	HTML5, CSS3, Bootstrap 5, Bootstrap Icons, vanilla JavaScript
Backend Framework	Python 3, Flask, Flask-Login, Flask-WTF
Machine Learning	PyTorch, TorchVision, NumPy, Pillow
Database	SQLAlchemy ORM; SQLite (default) / PostgreSQL (production)
Reporting	ReportLab (PDF generation)
Deployment	Gunicorn WSGI server, Docker-ready, Nginx reverse proxy

7.2 Core Application Modules

- app.py - Flask application factory and all HTTP routes (authentication, dashboard, upload/inference, history, PDF report).
- models.py - SQLAlchemy ORM models: User (credentials, role) and Prediction (image paths, predicted class, confidence, per-class distances/probabilities, notes, timestamps).
- forms.py - Flask-WTF forms for registration, login, and image upload with server-side validation.
- ml/network.py - The CNN embedding backbone (EmbeddingNet) shared by the Prototypical Network and Grad-CAM.
- ml/prototypical.py - Builds class prototypes from the support set and classifies a query image.
- ml/gradcam.py - Computes the gradient-based attention heatmap for the predicted class.
- ml/demo_support_set.py - Generates/loads the K-shot support set images per rare disease class.
- report.py - Assembles a downloadable PDF prediction report using ReportLab.

7.3 Database Design

Table	Key Columns	Purpose
users	id, full_name, email, password_hash, role, created_at	Stores authenticated user accounts
predictions	id, user_id (FK), original_image, heatmap_image, predicted_class, confidence, class_distances_json, class_probs_json, notes, created_at	Stores every analysis performed, linked to the user who ran it

8. Results and Discussion

The implemented system was validated end-to-end using automated functional tests covering account registration, login, image upload, few-shot inference, Grad-CAM heatmap generation, result rendering,

history listing, and PDF report download. All routes returned correct HTTP responses and the full pipeline - from a raw uploaded image to a stored prediction with an explanation heatmap and downloadable report - executed successfully within a few seconds per request on CPU-only hardware, confirming the design goal of a lightweight, GPU-independent inference path suitable for modest clinical deployment environments.

Qualitatively, the interactive upload interface (drag-and-drop with live preview, a loading overlay during inference, and an animated confidence ring and probability bars on the results page) provides a smooth clinician-facing workflow. The Grad-CAM overlay consistently highlighted spatially coherent regions of the input image, demonstrating that the metric-based adaptation of Grad-CAM (backpropagating prototype similarity rather than a classifier logit) produces meaningful, image-aligned attention maps even though the underlying encoder is a lightweight architecture.

Because the shipped support set is procedurally generated rather than sourced from real patients, the class probabilities reported by the demo system should be interpreted as evidence that the engineering pipeline functions correctly, not as clinically validated diagnostic accuracy. Section 10 discusses the steps required to move from this demonstration configuration to a clinically trained model.

9. Advantages of the Proposed System

- Requires only a handful of labelled images per rare disease class, unlike conventional CNNs that need thousands.
- New disease classes can be added by supplying a new K-shot support set, with no retraining of the encoder required.
- Grad-CAM explanations increase clinician trust by visually justifying each prediction.
- Full-stack design with authentication, history, and PDF reporting makes the system usable end-to-end, not just a research notebook.
- Lightweight CNN backbone keeps inference fast on CPU-only servers, easing deployment cost.
- Clean separation between the ML pipeline and the web application allows either to evolve independently.

10. Limitations and Future Scope

10.1 Limitations

- The bundled support set and CNN encoder are demonstration artefacts (synthetic images, untrained weights) rather than clinically validated components.
- Prototypical Networks assume a well-separated embedding space; without episodic training on a large, diverse ophthalmic dataset, embedding quality is not guaranteed to generalise to real-world image variation (lighting, camera type, patient demographics).
- Grad-CAM, like all gradient-based explanation methods, provides a coarse, approximate localisation and should not be treated as pixel-perfect lesion segmentation.
- The current system supports single-image analysis; it does not yet incorporate longitudinal (multi-visit) patient history or multi-modal data (OCT, angiography).

10.2 Future Scope

- Episodic meta-training of the CNN encoder on a large, diverse fundus dataset (e.g. combining public retinal datasets across many conditions) to learn a genuinely transferable embedding space.
- Collaboration with ophthalmology departments to curate real, IRB-approved K-shot support sets for each rare disease of interest, replacing the synthetic demo data.
- Extending the metric-based classifier to a Siamese/relation-network ensemble for improved robustness, and calibrating confidence scores against clinical outcomes.
- Adding role-based access control, audit logging, and DICOM/OCT support for deeper clinical integration.
- Deploying with GPU-backed inference and containerised horizontal scaling (Docker + Kubernetes) for multi-clinic use.

11. Conclusion

This project demonstrates that Few-Shot Learning, specifically Prototypical Networks, is a practical and architecturally clean approach to the fundamentally low-data problem of rare ophthalmic disease detection. By combining a metric-based few-shot classifier with a Grad-CAM-style explanation module and embedding the entire pipeline inside a secure, database-backed Flask web application with an interactive Bootstrap interface, the system delivers not just a machine learning proof of concept but a usable, extensible reference implementation. While the shipped configuration uses a synthetic demo support set for full out-of-the-box runnability, the architecture is deliberately designed so that real, clinically-sourced K-shot examples can be substituted directly, positioning the system as a solid foundation for a future clinically validated rare-disease screening tool.

References

[1] J. Snell, K. Swersky, and R. Zemel, "Prototypical Networks for Few-shot Learning," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[2] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," in IEEE International Conference on Computer Vision (ICCV), 2017.

[3] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra, "Matching Networks for One Shot Learning," in Advances in Neural Information Processing Systems (NeurIPS), 2016.

[4] G. Koch, R. Zemel, and R. Salakhutdinov, "Siamese Neural Networks for One-shot Image Recognition," ICML Deep Learning Workshop, 2015.

[5] C. Finn, P. Abbeel, and S. Levine, "Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks," in International Conference on Machine Learning (ICML), 2017.

[6] V. Gulshan et al., "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs," JAMA, 2016.

[7] Flask Documentation, Pallets Projects. [Online]. Available: https://flask.palletsprojects.com

[8] PyTorch Documentation, PyTorch Foundation. [Online]. Available: https://pytorch.org/docs